INPUT

Today's Topics

- Input events
- State machines
- Event dispatch & propagation

Raw Input Events

- The usual input hardware has state:
  - ~100 keys on the keyboard (down or up)
  - (x,y) mouse cursor position on the screen
  - one, two, or three mouse buttons (down or up)
- A "raw" input event occurs when this state changes

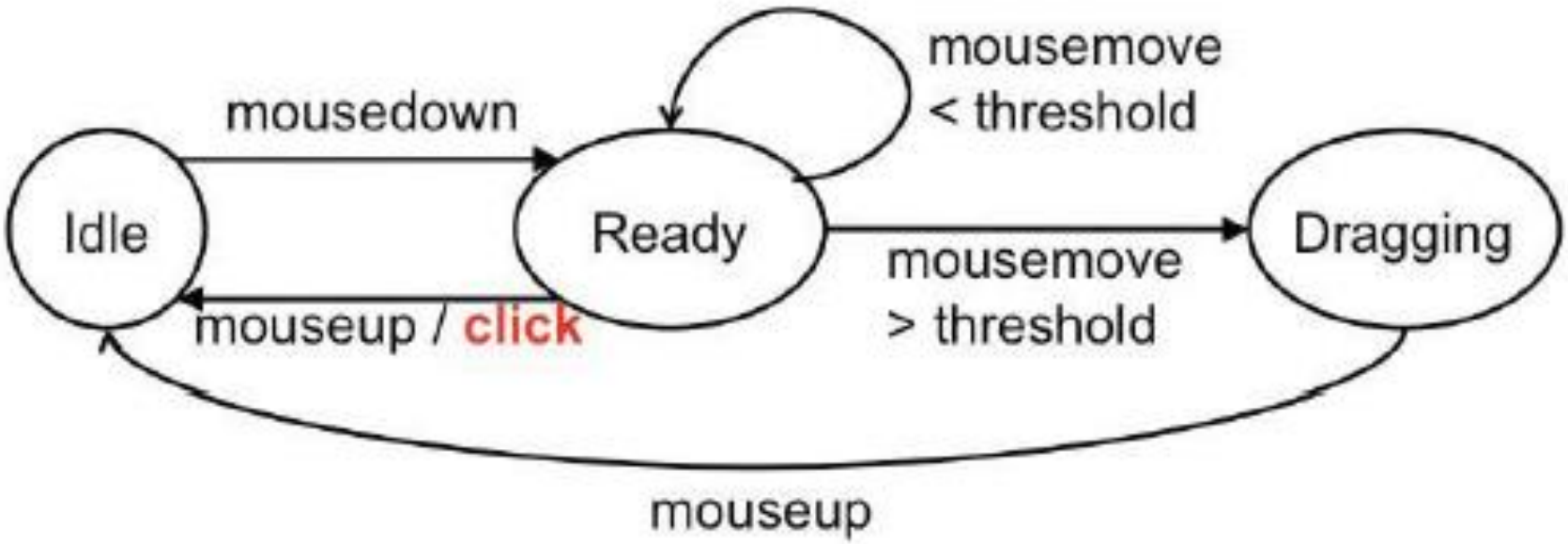
	jQuery event
– key pressed or released	`keydown, keyup`
– mouse moved	`mousemove`
– button pressed or released	`mousedown, mouseup`

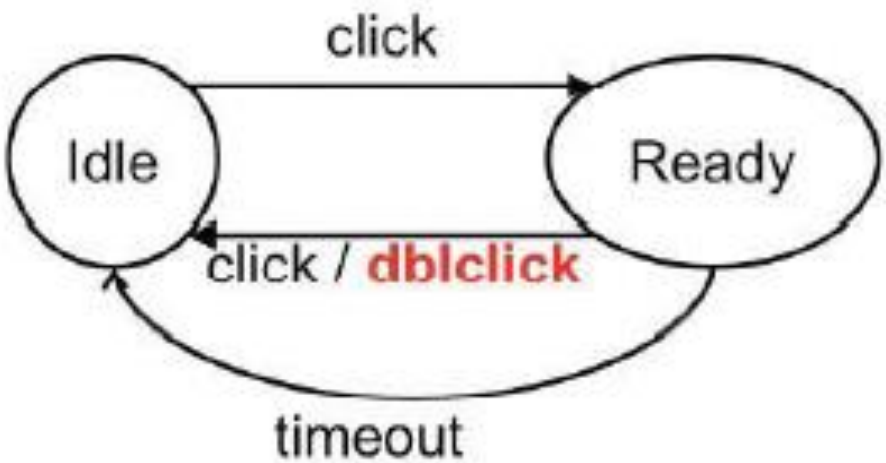
Translated Events

- Raw events are translated into higher-level events

	jQuery event
– Clicking	`click`
– Double-clicking	`dblclick`
– Character typed	`keypress`
– Entering or exiting an object's bounding box	`mouseenter`, `mouseleave`

State Machines Translate Events





```
<html>
<head>
<script src="http://code.jquery.com/jquery-1.5.min.js"></script>
<script>
$(function() {

$("#A").mousedown(function() { $("#A").append("A down<br>") })

$("#A").mouseup(function() { $("#A").append("A up<br>") })

$("#A").click(function() { $("#A").append("A clicked<br>") })

$("#A").dblclick(function() { $("#A").append("A double-clicked<br>")
})
})
</script>
</head>
<div id="A"
style="width:100px;height:100px;background:lightYellow;padding:5
px">
</div>
</html>
```

A down
A up
A clicked
A down
A up
A clicked
A double-
clicked

Keyboard Focus

- An object in the view tree has the keyboard focus

	jQuery event
– Keyboard focus gained or lost	`focusin`, `focusout`

- Changing keyboard focus
  - by user input event: e.g. mouse down, Tab
  - programmatically by a method call
- Not all HTML elements can have keyboard focus by default

   to force ability to take focus

On some X Windows window managers, you can configure the keyboard focus to follow the mouse pointer

Properties of an Input Event

- Mouse position (X,Y)
- Mouse button state
- Modifier key state (Ctrl, Shift, Alt, Meta)
- Timestamp
  - Why is timestamp important?

Double-clicking etc...

Events are stored in a queue, and an event may languish in the queue for an uncertain interval until the application actually handles it, so it’s necessary for the time of the event to be captured as close to the event’s actual occurrence

- Keyboard key, character, or mouse button that changed
  - jQuery event.which overloads this for mouse events and key events

Different keyboard hardware with different sets of keys

In HTML/Javascript, different browsers may work differently

Translated key events do not represent a keystroke (like Shift or PgUp or the A key), but rather a character (like “a” or “A” or “%”)

Keystrokes are identified by physical keys on the keyboard; characters are identified by values in a character set (like Unicode or ASCII)

```
<html>
<head>
<script src="http://code.jquery.com/jquery-1.5.min.js"></script>
<script>
$(function() {
$("#A").mousedown(function() { $("#A").append("A down<br>") })
$("#A").mouseup(function() { $("#A").append("A up<br>") })
$("#A").click(function() { $("#A").append("A clicked<br>") })
$("#A").dblclick(function() { $("#A").append("A double-clicked<br>") })

$("#b").keydown(function(event) { $("#A").append("A down " + event.which
+ " / ") })

$("#b").keyup(function(event) { $("#A").append("A up " + event.which + " / ") })

$("#b").keypress(function(event) { $("#A").append("A press " + event.which + " / ") })
})

</script>
</head>
<div id="A" style="width:1000px;height:100px;background:lightYellow;padding:5px">
</div>
<textarea id="b"></textarea>
</html>
```

A down 65 / A press 97 / A up 65 / A down 66 / A press 98 / A up 66

ab

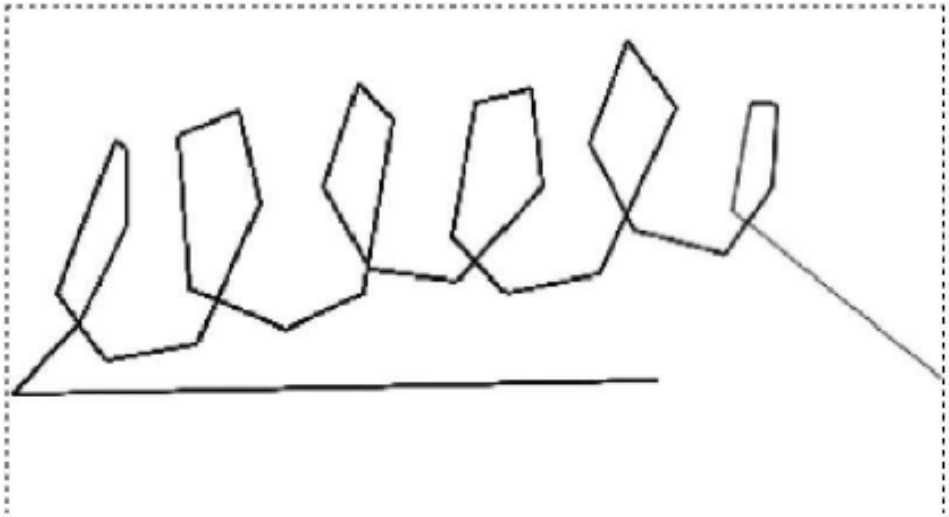
Event Queue

- Events are stored in a queue
  - User input tends to be bursty
  - Queue saves application from hard real time constraints (i.e., having to finish handling each event before next one might occur)
- Mouse moves are coalesced into a single event in queue
  - If application can't keep up, then sketched lines have very few points

you're dragging a big object across the screen, and the application can't repaint the object fast enough to keep up with your mouse, you don't want the mouse movements to accumulate in the queue, because then the object will lag behind the mouse pointer, diligently (and foolishly) following the same path your mouse did

you're sketching a freehand stroke with the mouse, and some of the mouse movements are coalesced, then the stroke may have straight segments at places where there should be a smooth curve

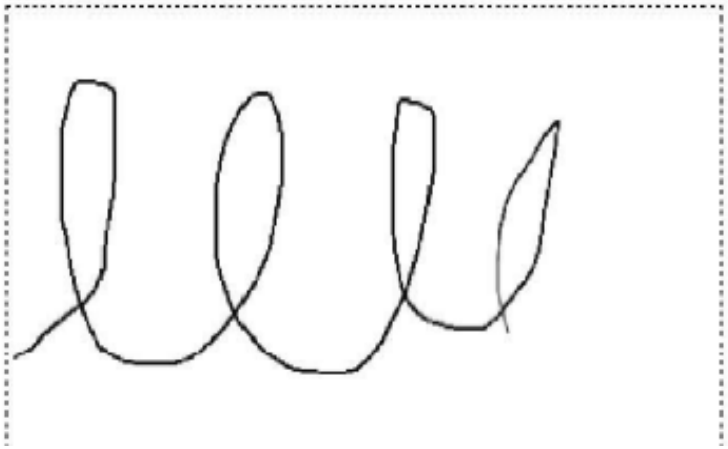
```
<script src="http://code.jquery.com/jquery-1.5.min.js"></script>
<canvas id="A" width="400" height="300" style="border: 1px dashed"></canvas>
<script>
$(function() {
var ctx = $("#A").get(0).getContext("2d")
var pos = $("#A").offset()
ctx.translate(-pos.left, -pos.top) // move origin of canvas to origin of document, so
we can use event.pageX/pageY directly
$("#A").mousemove(function(event) { sleep(0) //100
 ctx.lineTo(event.pageX, event.pageY); ctx.stroke
() })
function sleep(ms) { var wake = now() + ms; while (now() < wake) { } }
function now() { return new Date().getTime() }
})
</script>
```



Without coalescing

Event coalescing can't be easily disabled in HTML/Javascript, so we'll simulate what life would be like without coalescing by creating our own queue of mouse points (queue and pushEvent() in the code below) and an event handler that reads from it (eventLoop())

```
<script src="http://code.jquery.com/jquery-1.5.min.js"></script>
<canvas id="A" width="400" height="300" style="border: 1px dashed"></canvas>
<script>
$(function() {
var ctx = $("#A").get(0).getContext("2d")
var pos = $("#A").offset()
ctx.translate(-pos.left, -pos.top) // move origin of canvas to origin of document, so we can use
event.pageX/pageY directly
$("#A").mousemove(function(event) { pushEvent(event.pageX, event.pageY) })
var queue = [];
var DELAY = 1;
function pushEvent(x,y) {
if (!queue.length) setTimeout(eventLoop, DELAY)
queue.push({x:x, y:y})
}
function eventLoop() {
if (!queue.length) return;
var pt = queue.shift();
ctx.lineTo(pt.x, pt.y);
ctx.stroke();
setTimeout(eventLoop, DELAY)
}
})
</script>
```



Event Loop

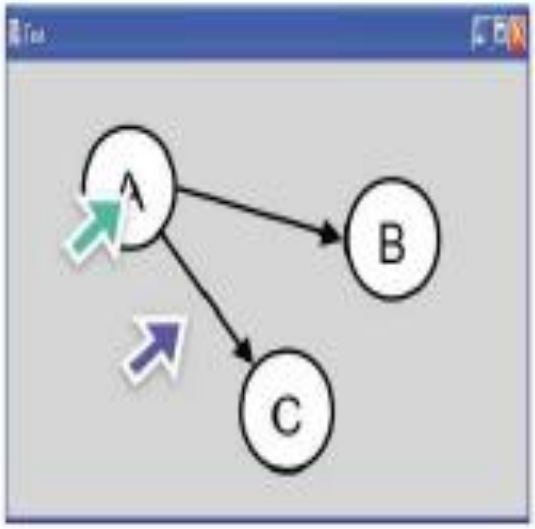
- While application is running
  - Block until an event is ready
  - Get event from queue
  - Translate raw event into higher-level events
    - Generates double-clicks, characters, focus, enter/exit, etc.
    - Translated events are put into the queue
  - Dispatch event to target component
- Who provides the event loop?
  - High-level GUI toolkits do it internally (Java Swing, VB, C#, HTML)
  - Low-level toolkits require application to do it (MS Win, Palm, Java SWT)

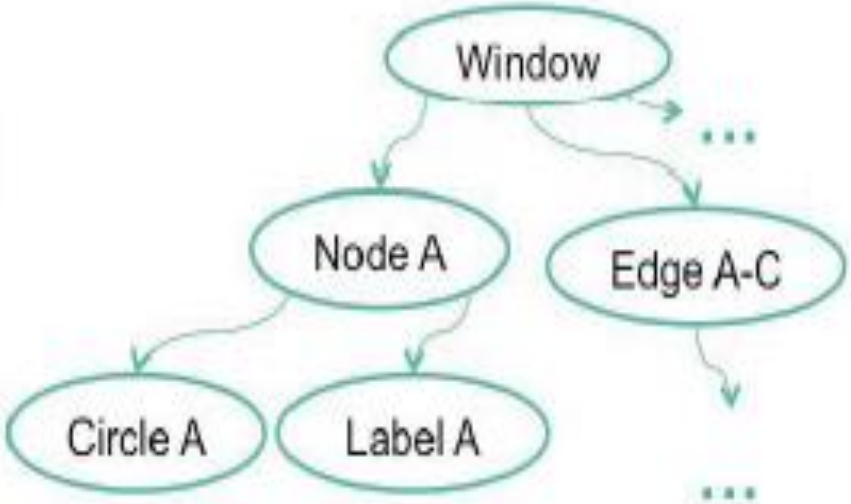
event loop reads events from the queue and dispatches them to the appropriate components in the view hierarchy
On some systems (notably Windows), the event loop also includes a call to a function that translates raw events into higher-level ones
On most systems, translation happens when the raw event is added to the queue, not when it is removed

Event Dispatch & Propagation

- Dispatch: choose target component for event
  - Key event: component with keyboard focus
  - Mouse event: component under mouse (**hit testing**)
    - **Mouse capture**: any component can grab mouse temporarily so that it receives all mouse events (e.g. for drag & drop)
- Propagation: event bubbles up hierarchy
  - If target component doesn't handle event, the event passes up to its parent, and so on up the tree
- Consumption: event stops propagating
  - May be automatic (because some component finally handles it) or manual (keeps going unless explicitly stopped)

Hit Testing and Event Propagation





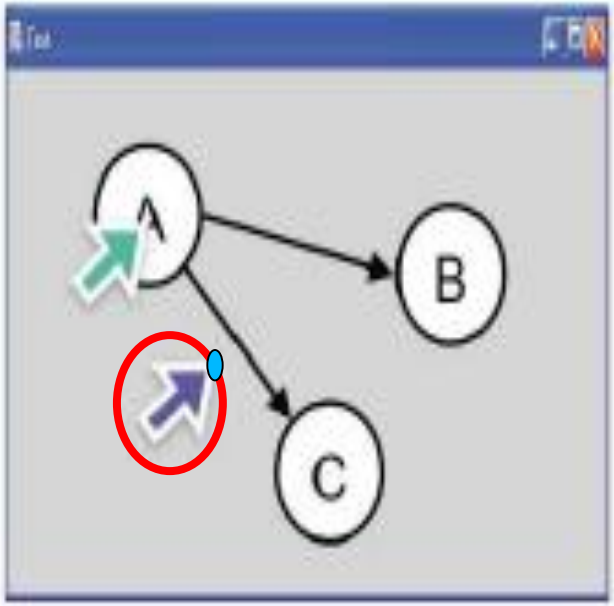
topmost component whose bounding box contains that point is Label A

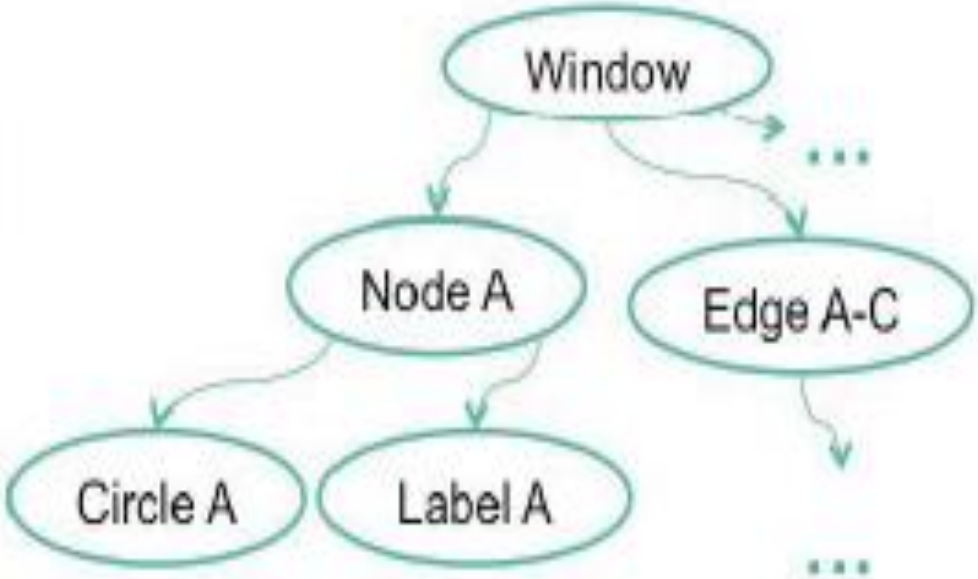
If Label A doesn't handle the mouse-move event, then the event is propagated up to Node A

if that doesn't handle the event either, it's propagated to Window, and then discarded

Circle A never sees the event, because event propagation goes up the tree, not down through z-order layers

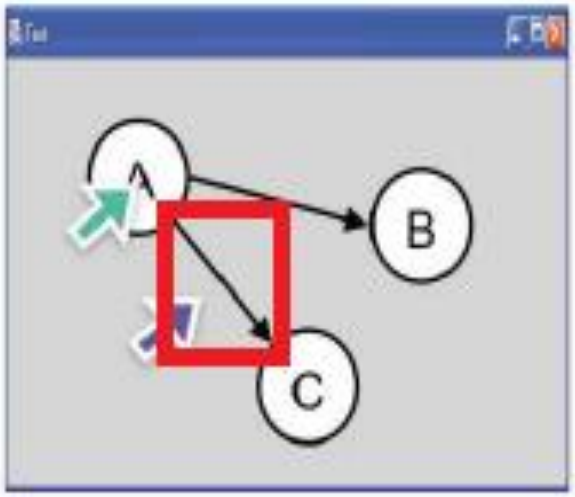
Hit Testing and Event Propagation

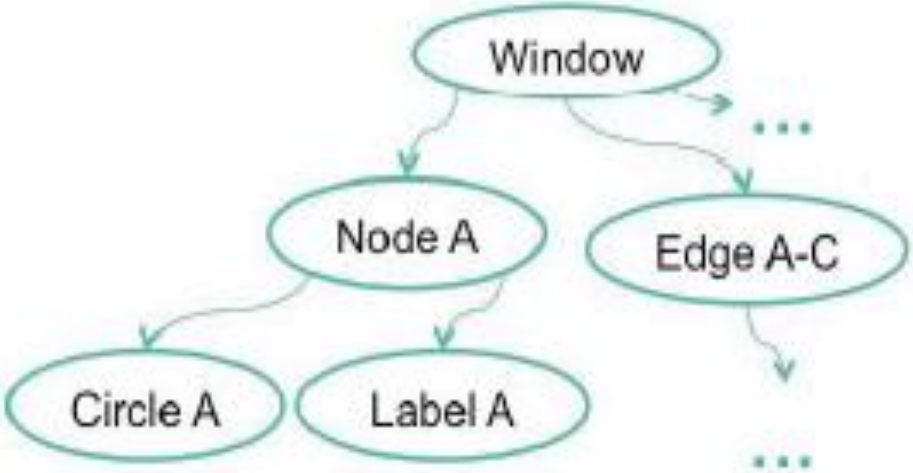




?

Hit Testing and Event Propagation





What component will be the initial target for a mouse-move event for this point?
answer depends on how hit-testing is done by the toolkit
Some toolkits support only rectangular bounding-box hit testing, in which case Edge A-C will be the event target
Other toolkits allow hit testing to be overridden and controlled by components themselves, so that Edge A-C could test whether the point actually falls on (or within some small threshold of) the actual line it draws. Java Swing supports this by overriding Component.contains(). If Edge A-C rejects the point, then the next component in z-order whose bounding box contains the mouse position is the window itself, so the event would be dispatched directly to the window.

HTML code for experimenting with event dispatch and propagation

```
<html>
<head>
<script src="http://code.jquery.com/jquery-1.5.min.js"></script>
<script>
$(function() {
$("#B").bind("mousedown mouseup mousemove", function(event) {
$("#scr").append(this.id + event.target.id + event.type + "   ") })
$("#A").bind("mousedown mouseup mousemove", function(event) {
$("#scr").append(this.id + event.target.id + event.type + "   ") })
$(window).bind("mousedown mouseup mousemove", function(event) {
$("#scr").append("window" + event.target.id + event.type + "   ") })
})
</script>
</head>
<div id="A"
style="width:100px;height:100px;background:lightYellow;padding:5px">A
<div id="B" style="margin:20px; padding:5px; background:lightPink;">B</div>
</div>
<div id="scr"
style="width:1200px;height:400px;background:lightYellow;padding:5px">
</div>
</html>
```

A

B

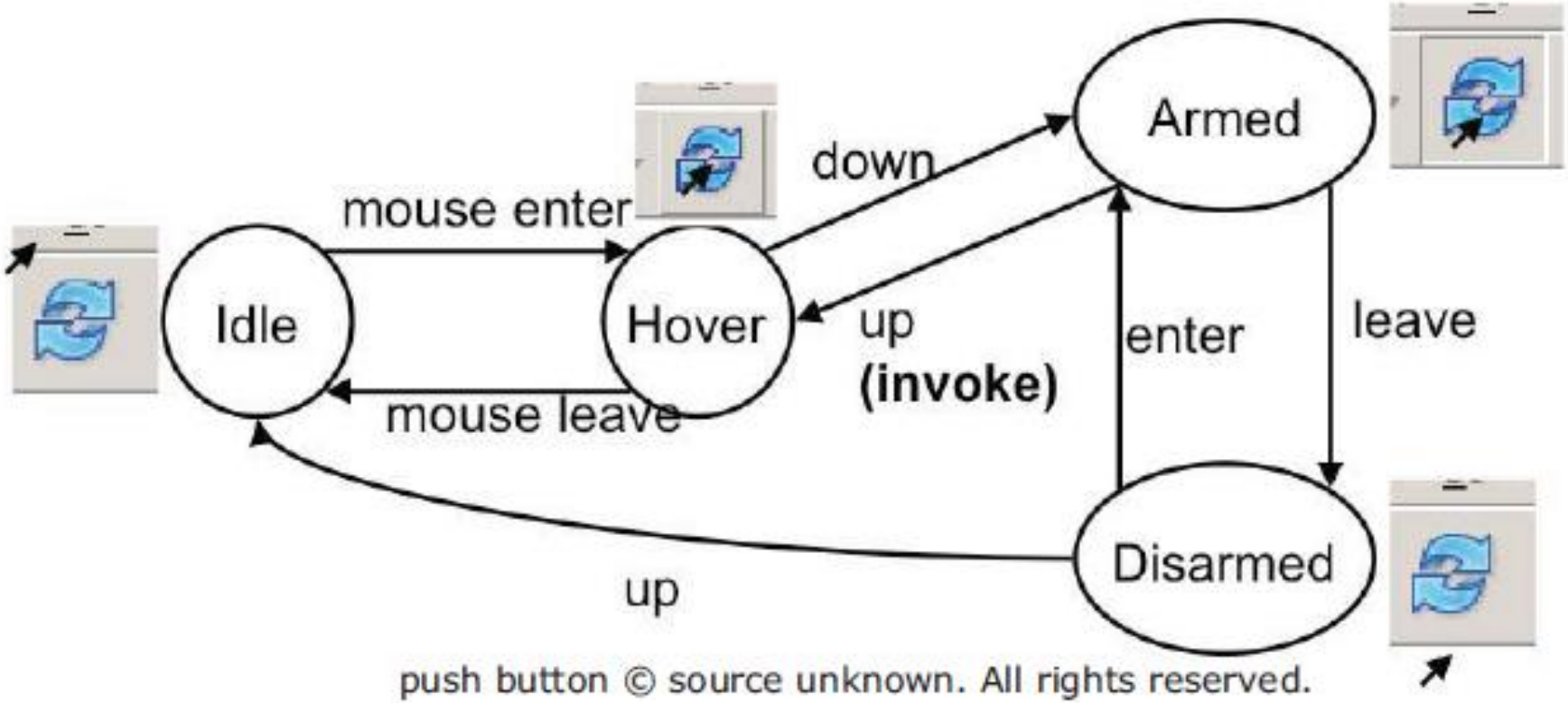
windowmousemove windowmousemove windowmousemove windowmousemove windowmouse
windowAmousemove AAmousemove windowAmousemove AAmousemove windowAmousemov
AAmousemove windowAmousemove AAmousemove windowAmousemove AAmousemove win
windowAmousemove AAmousemove windowAmousemove AAmousemove windowAmousemov
AAmousemove windowAmousemove AAmousemove windowAmousemove AAmousemove win
windowBmousemove BBmousemove ABmousemove windowBmousemove BBmousemove ABm
BBmousemove ABmousemove windowBmousemove BBmousemove ABmousemove windowBm
ABmousemove windowBmousemove BBmousemove ABmousemove windowBmousemove BBm
windowBmousemove BBmousemove ABmousemove windowBmousemove BBmousedown ABm
BBmousemove ABmousemove windowBmousemove BBmousemove ABmousemove windowBm
ABmouseup windowBmouseup

Javascript Event Models

- Events propagate in different directions on different browsers
  - Netscape 4: downwards from root to target
  - Internet Explorer: upwards from target to root
  - W3C standardized by combining them: first downwards ("capturing"), then upwards ("bubbling")
    - Firefox, Opera, Safari

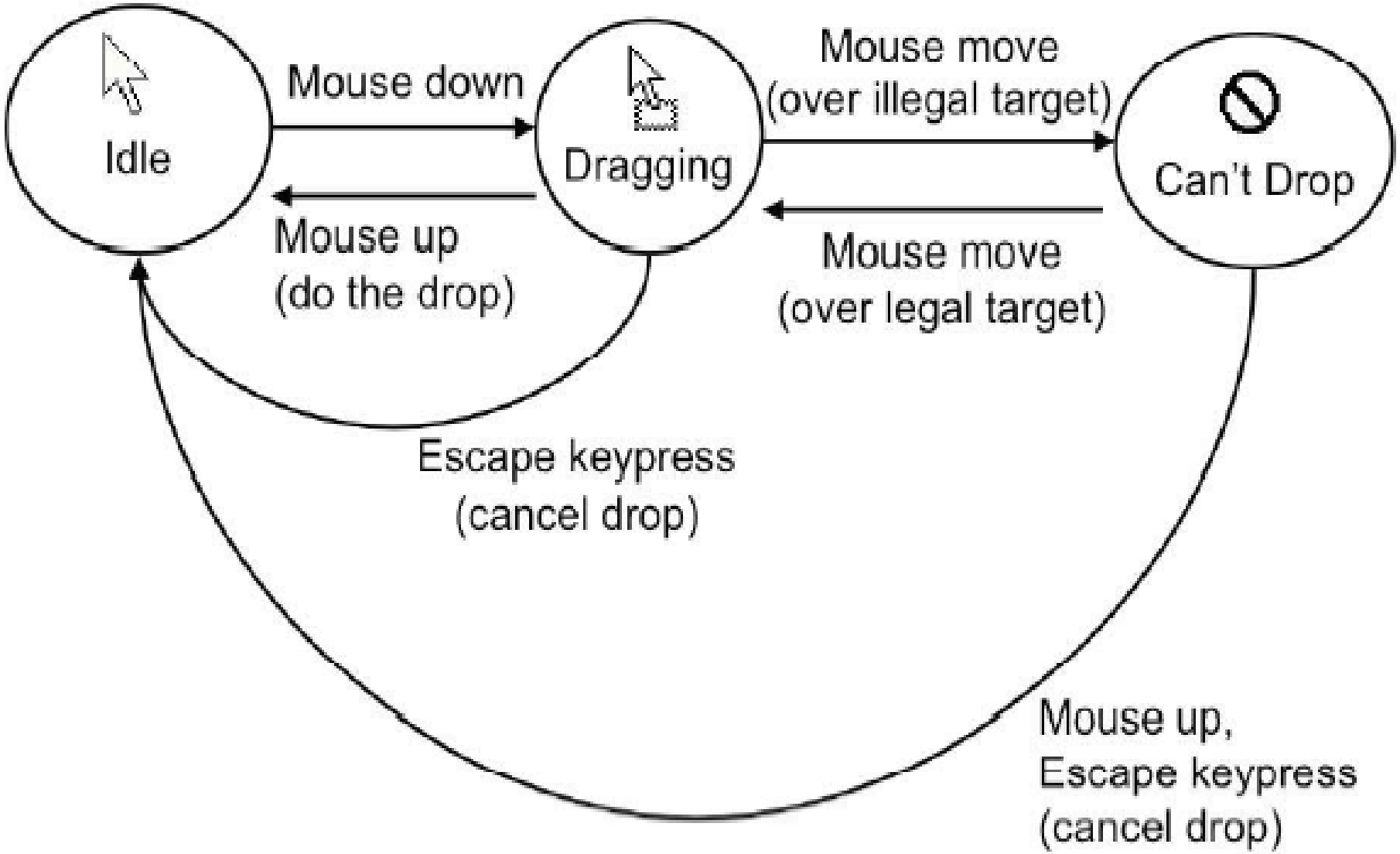
Designing a Controller

- A controller is a finite state machine
- Example: push button



push button © source unknown. All rights reserved.

Another Finite State Machine



Summary

- Input events are raw and translated
- Events are **dispatched** to a target view and **propagated** up (or down then up) the view hierarchy
- State machines are a useful pattern for thinking about input